Assignment 1

RAG Assignment: Baseline vs. Re-Ranking

Language Models and Agents — Monsoon 2026

Aditya Peketi - 2024122001



INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY

HYDERABAD

International Institute of Information Technology, Hyderabad

Contents

1 Setup and Environment **3**

2 Architecture Comparison: **3**

2.1 Bi-encoder: 3

2.2 Cross-encodere: 3

3 Qualitative and rank shift analysis **3**

3.1 Reasoning and findings: 5

3.2 Category-Specific Explanations (Why the Bi-encoder Failed) 5

3.3 Document-Specific Failures 5

3.4 Control Cases: Where Re-ranking Does Not Help 5

4 Latency Profiling: **6**

5 Scaling: Why Not Use the Cross-Encoder on the Whole Corpus? **6**

1 Setup and Environment

- **Github Link -** https://github.com/adityapeketii/lma-RAG
- **LLM Used -** Gemini API
- **Dataset -** I picked up a Game rulebook because I like games so why not. The rulebook I picked was **Magic: The Gathering Comprehensive Rules** (Sept 2026) because I felt like it is a good choice because it's long and heavily cross-referenced, and full of *except, unless* and *can't* clauses. That gives a retriever plenty of chances to fail.

2 Architecture Comparison:

2.1 Bi-encoder:

- The query and each chunk are encoded independently into one fixed vector each (384 dimensions for MiniLM).
- Relevance is the cosine similarity between them
- The biencoder is fast because chunk vectors are computed once, offline. A query then needs one encode plus a vector search.
- Even though the Bi-encoder is fast I feel like it is not so accurate because the whole passage is compressed into a single vector before the question is known. Qualifiers ("if any", "can't", "unless") get blurred, and shared keywords dominate.

2.2 Cross-encodere:

- In the case of Cross-encoder both the query and the chunks are fed together in one sequence. Self-attention lets every query token attend to every chunk token in every layer. Basically the output is a single relavance score and not a single vector.
- I also feel like the Cross-encoder is slow but more accurate because it judges the actual pair, so it can match "required?" against "if any". But nothing can be precomputed, so it needs one forward pass per (query, chunk) pair at query time and this is the reason it is kinda slow. We can also observe this through the below examples.

Prompt:

Both pipelines use the exact same prompt. It asks the LLM to act as a Magic: The Gathering rules judge, to answer only from the rules excerpts it is given, to cite the rule numbers it uses (this is only for our evaluation of the outputs), and to say so if the excerpts do not contain enough information. The model (Gemini, temperature which I set to temp 0) and the prompt are the same for both pipelines, so the only thing that changes is which 3 chunks go into the prompt. The difference between Pipeline 1 and Pipeline 2 is that Pipeline 1 gets the top 10 relevant chunks and then just send the top 3 while Pipeline 2 would re rank the top 10 and then sent top 3 after re ranking.

3 Qualitative and rank shift analysis

Table 1: Rank shift table

Category	**Query**	**Target (chunk)**	**Baseline rank**	**Re-ranked rank**
Keyword trap	Can I respond to my opponent playing a land?	116.2a (116.2)	#9	#3
Keyword trap	Does an attacking creature stay attacking if it stops being a creature?	506.4 (506.4)	#7	#3
Exception	Does deathtouch kill a creature with indestructible?	702.12b (702.12)	#5	#3
Exception	Is there ever a situation where players get priority during cleanup?	514.3a (514.3)	#5	#1
Negation	Is the defending player required to block?	509.1a (509.1)	#9	#1
Negation	Can I pay part of a cost if I don't have enough mana?	118.3 (118.3)	#7	#1
Buried chunk	Can a player concede at any time?	104.3a (104.3)	#8	#3
Buried chunk	Do I lose immediately when my library is empty?	121.4 (121.4)	#5	#2

Table 2: Answer 1 vs Answer 2

Query	**Baseline top 3**	**Answer 1**	**Answer 2**	**Failure type**
Land	305.3, 305.4, 305.1	Correct (305.1 already says you can't respond)	Correct, adds 116.2a	None: retrieval improved, answer unchanged
Attacking creature	glossary:Attacking Creature, 508.4, 508.4#2	"Not enough information"	Correct ("No", 506.4)	**Omission**
Deathtouch	glossary:Deathtouch, 702.2, glossary:Lethal Damage	"Not enough information" (knew deathtouch destroys; nothing on indestructible)	Correct ("No", 702.12b)	**Omitted exception clause**
Cleanup	500.3, glossary:Pass in Succession, 117.3	"Yes, but can't say when"	Gives the 514.3a conditions	**Incomplete answer** (omitted clause)
Block	glossary:Block, 805.10#3, glossary:Defending Player	"Not enough information"	Correct (quotes "if any")	**Omission**
Partial payment	glossary:Alternative Cost, glossary:Additional Cost, glossary:Mana	"Not enough information"	Correct (118.3 + 601.2h)	**Omission**
Concede	723.6, glossary:Concede, 101.1	Correct	Correct, cites 104.3a	None: the rulebook repeats this fact
Empty library	401.6, 701.24, 401.1	"Not enough information"	Correct (you lose only on an attempted draw), plus one off-topic 727.3 note	**Omission**

3.1 Reasoning and findings:

In the baseline evaluation, the model exhibited no hallucinations and produced no misstated rules (theres a reason tho which is the design of the prompt). Instead, the primary failure mode was omission: whenever the crucial retrieval chunk fell outside the top three results, the model either stated that there was "not enough information" or provided a partial answer. This conservative behavior directly stems from the prompt's strict constraints, which instruct the model to answer exclusively using the provided excerpts and to explicitly admit when those excerpts are insufficient. This is the sole reason why I observed no hallucinations or misstated rules.

Overall, the retrieval improvements yielded positive results, with **six of the eight** answers improving and the remaining two already being correct in the baseline. It is important to note the value of the strict prompting strategy; without the explicit instruction to rely only on the retrieved text, **a weaker prompt might have encouraged the model to fill knowledge gaps using its intrinsic training data** (technically hallucinations and misstated rules). While this might have reduced omissions, it would have introduced a significant risk of generating misstated rules. Plis refer the python notebook for a detailed explanations and more examples

3.2 Category-Specific Explanations (Why the Bi-encoder Failed)

- **Keyword trap:** The question uses words (like "land" or "attacking creature") that match incorrect text, while the true answer uses completely different words (like "special action").
- **Exception:** General rules score higher than their exceptions, which are often hidden in different sections.
- **Negation:** Small but important words (like "if any" or "partial") barely change the search scores. Larger keywords (like "cost" or "block") take over the search results instead.
- **Buried chunk:** The correct answer is hidden inside a long, multi-topic paragraph, or it is located in a completely different section from where the search words point.

3.3 Document-Specific Failures

1. **Glossary entries are distracting:** Glossary items are short and repeat key terms, which makes them score artificially high in the baseline search (appearing in 6 of 8 top results). The re-ranker correctly pushes them down the list.
2. **Multiplayer rules distract:** Rules for special game types or multiplayer games rank high for basic two-player questions simply because they repeat common terms like "defending player."
3. **Long chunks hide facts:** Specific rules get diluted and lost when they are buried inside very long blocks of text.

3.4 Control Cases: Where Re-ranking Does Not Help

Case	Query	Target	Result
Re-ranker regression	Can a counterspell counter an ability?	701.6a	Rank drops from #1 to #4. The re-ranker preferred texts about physical "counters" (markers); the final answer lost important details.
Recall miss	Is a creature with 0 toughness destroyed?	704.5f	Ranked #82 of 2,324. Outside the top 10, so the re-ranker never sees it; the model answers "not enough information."

Table 3: Control cases where re-ranking does not help.

4 Latency Profiling:

Stage	Mean (ms)	Median (ms)	p95 (ms)
Pipeline 1: bi-encoder top 10	9.40	9.44	12.21
Cross-encoder re-rank (10 pairs)	281.64	285.16	350.33
Pipeline 2: bi-encoder + re-rank	**291.04**	**294.81**	**359.87**

Table 4: Retrieval vs. re-ranking latency

Pipeline	Mean (ms)	Median (ms)	Max (ms)
Pipeline 1 (retrieve + LLM)	14,987	10,866	32,898
Pipeline 2 (retrieve + re-rank + LLM)	21,385	12,432	66,309

Table 5: End-to-end latency including the Gemini call.

From my experiments we can see that Re-ranking the top 10 candidates adds about 282 ms per query, but this is negligible next to the LLM call, which takes about 11–12 s (whixh is the median) and makes re-ranking only about 2–3% of the end-to-end latency (Tables 4 and 5). The larger mean and maximum for Pipeline 2 come from network variance and one call that included rate-limit retries (66 s), not from re-ranking, so the medians are the fairer comparison. LLM times were recorded when each answer was first generated, and the one-time cost of embedding the corpus (28.6 s on CPU) is paid offline rather than per query.

5 Scaling: Why Not Use the Cross-Encoder on the Whole Corpus?

To answer this, I ran the cross-encoder on all 2,324 chunks for a single query and compared it with re-ranking only the top 10 candidates and with the bi-encoder search (Table 6).

Method	Items scored	Seconds per query
Cross-encoder over full corpus	2,324	40.77
Cross-encoder over top 10	10	0.23
Bi-encoder search over full corpus	2,324	0.0063

Table 6: Measured time for one query (CPU).

In my case scoring the full corpus with the cross encoder took about 41 s for one query. This is about 177× slower than re-ranking the top 10, and very much slower than the bi-encoder search. Since each (query, chunk) pair costs about 17.5 ms, the time grows linearly with the size of the corpus which is O(N) (Table 7).

The main reason for this difference is that the cross-encoder cannot do any work in advance. Its score depends on both the query and the chunk together, so every new query needs one full model pass for every chunk. The bi-encoder, on the other hand, embeds all chunks only once, offline. At query time it only embeds the query and compares one vector against the stored vectors, which took about 6 ms here. With approximate nearest-neighbour indexes (for example FAISS IVF or HNSW), this search can scale to millions of chunks.

Because of this, the two models are best used together. The bi-encoder is used for *recall*: it quickly narrows thousands of chunks down to a small candidate list. The cross-encoder is then used for *precision*: it carefully re-orders only those few candidates. This gives most of the accuracy of the cross-encoder at a small, fixed extra cost.

The limitation of this design is that the re-ranker can only re-order what the bi-encoder has already found. For example, in the query “Is a creature with 0 toughness destroyed?”, the correct rule (704.5f) was

Corpus size	Estimated cross-encoder time per query
1,000	$\approx$ 17.5 s
100,000	$\approx$ 1,754 s ($\approx$ 29 minutes)
10,000,000	$\approx$ 175,445 s ($\approx$ 49 hours)

Table 7: Estimated cross-encoder cost for larger corpora (about 17.5 ms per pair, linear in corpus size).

ranked #82 by the bi-encoder, so it never reached the top 10 and re-ranking could not recover it. Retrieving more candidates (for example 20–50 instead of 10) would reduce such misses, at a cost of roughly 28 ms per extra candidate on CPU.